The Superior University, Lahore

LAB-TASK (5) (Spring-2026)

Course Title:	Programming for AI (Lab)				Course Code:	-	Credit Hours	1
Instructor:	Sir Rasikh				Program:	BSDS		
Semester:	4th	Batch:	Morning	Section:	4A	Date:	-	
Time Allowed:	-				Maximum Marks:		N/A	
Student's Name:	Shumaila Maryam				Roll No.	SU92-BSDSM-S25-062		

Work Assigned:

Task 1; Implement the main OpenCV functions in a Jupyter Notebook (.ipynb) (main topics, can skip others) (https://www.geeksforgeeks.org/opencv-python-tutorial/)

DEPARTMENT OF SOFTWARE ENGINEERING

Table of Contents

- 1. Image Processing and Enhancement
- 1.1 Image Blur (Gaussian, Median, Bilateral)
- 1.2 Grayscaling
- 1.3 Scaling, Rotating, Shifting
- 1.4 Edge Detection
- 1.5 Normalization
- 2. Image Segmentation and Thresholding
- 2.1 Simple Thresholding Methods
- 2.2 Adaptive Thresholding
- 2.3 Otsu Thresholding
- 3. Morphological Operations & Filtering
- 3.1 Erosion and Dilation
- 3.2 Opening and Closing
- 3.3 Bilateral Filtering
- 3.4 Image Denoising
- 4. Advanced Image Manipulation & Background Subtraction
- 4.1 Image Inpainting
- 4.2 Background Subtraction
- 4.3 Foreground Extraction (GrabCut)
- 5. Feature Detection and Description
- 5.1 Line Detection (Hough Lines)
- 5.2 Corner Detection (Harris, Shi-Tomasi)

1. IMAGE PROCESSING AND ENHANCEMENT

Image processing and enhancement techniques are fundamental to computer vision. These operations improve image quality, prepare images for further analysis, and extract meaningful information. This section covers essential preprocessing techniques including blur filters, color space conversions, geometric transformations, and edge detection.

1.1 Image Blur Techniques

Image blurring is used to reduce noise, smooth images, and prepare data for further processing. Different blur techniques work in different ways:

Figure 1.1a: Original Image

Gaussian Blur

Gaussian blur uses a Gaussian filter kernel to smooth an image by averaging neighboring pixels. This is the most commonly used blur technique.

Formula: $G(x,y) = 1/(2\pi\sigma^2) \times e^{(-(x^2+y^2)/(2\sigma^2))}$

Parameters: Kernel size (15×15), Standard deviation $\sigma=0$

Benefits: Reduces noise, smooths transitions, preserves edges better than simple averaging

Code:

```
Gaussian = cv2.GaussianBlur(resized_image, (15, 15), 0)
Gaussian_rgb = cv2.cvtColor(Gaussian, cv2.COLOR_BGR2RGB)
plt.imshow(Gaussian_rgb)
plt.title('Gaussian Blurred Image')
plt.axis('off')
plt.show()
```

✓ 0.1s

Median Blur

Median blur replaces each pixel with the median value of neighboring pixels. Excellent for salt-and-pepper noise removal.Kernel size: 15×15 pixelsBenefits: Very effective for salt-and-pepper noise, preserves edges wellUse case: Medical imaging, satellite imagery with speckle noise

Code:

```
median = cv2.medianBlur(resized_image, 15)
median_rgb = cv2.cvtColor(median, cv2.COLOR_BGR2RGB)

plt.imshow(median_rgb)
plt.title('Median Blurred Image')
plt.axis('off')
plt.show()
```

✓ 0.2s

Bilateral Blur

Bilateral filter smooths while preserving edges by considering both spatial distance and pixel intensity differences.

Parameters: Diameter=15, Sigma Color=175, Sigma Space=175

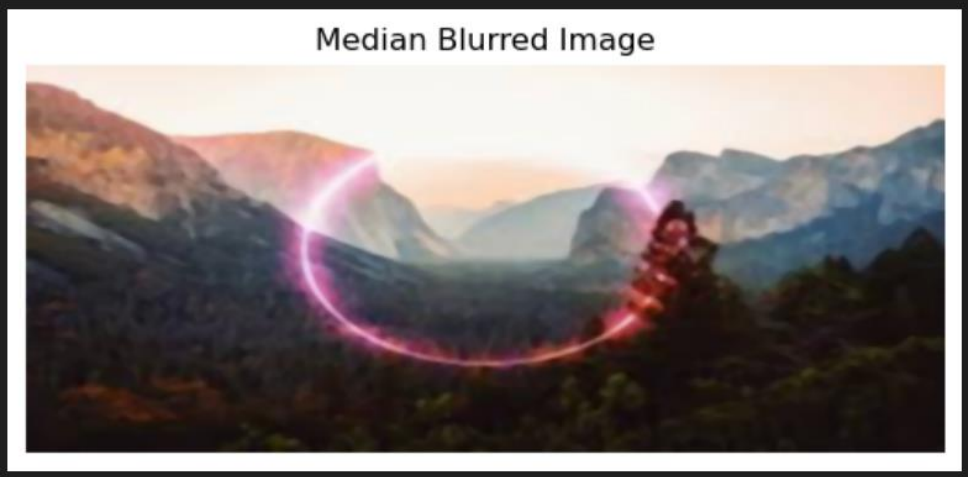
Benefits: Edge-preserving blur, effective for denoising while maintaining sharp boundaries

Use case: Photo enhancement, detailed image smoothing

```
median = cv2.medianBlur(resized_image, 15)
median_rgb = cv2.cvtColor(median, cv2.COLOR_BGR2RGB)

plt.imshow(median_rgb)
plt.title('Median Blurred Image')
plt.axis('off')
plt.show()
```

✓ 0.2s



1.2 Grayscaling

Grayscaling converts a color image to grayscale by combining RGB channels. This reduces computational complexity and is essential for many computer vision algorithms.

Standard conversion formula: Gray = 0.299×R + 0.587×G + 0.114×B

Benefits: Reduces data size by 3x, faster processing, suitable for edge detection and thresholding

Methods used in implementation:

cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) - Color space conversion

cv2.imread(path, 0) - Direct grayscale loading

```
import cv2

image = cv2.imread('Images/Task 5.jpg')

gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

cv2.imshow('Grayscale', gray_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

✓ 4.0s

```
import cv2

img = cv2.imread('Images/Task 5.jpg', 0)

cv2.imshow('Grayscale Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

✓ 1.5s

1.3 Geometric Transformations

Geometric transformations modify image size and orientation:

Image Scaling Enlarging (3.0x) and shrinking (1/3.0x) images using interpolation methods Interpolation methods:cv2.INTER_CUBIC: High-quality enlargement using cubic interpolationcv2.INTER_AREA: Efficient shrinking with pixel area relation

```
image = cv2.imread('Images/Task 5.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
scale_factor_1 = 3.0
scale_factor_2 = 1/3.0
height, width = image_rgb.shape[:2]
new_height = int(height * scale_factor_1)
new_width = int(width * scale_factor_1)

zoomed_image = cv2.resize(src =image_rgb,
                          dsize=(new_width, new_height),
                          interpolation=cv2.INTER_CUBIC)

new_height1 = int(height * scale_factor_2)
new_width1 = int(width * scale_factor_2)
scaled_image = cv2.resize(src= image_rgb,
                          dsize =(new_width1, new_height1),
                          interpolation=cv2.INTER_AREA)

fig, axs = plt.subplots(1, 3, figsize=(10, 4))
axs[0].imshow(image_rgb)
axs[0].set_title('Original Image Shape:'+str(image_rgb.shape))
axs[1].imshow(zoomed_image)
axs[1].set_title('Zoomed Image Shape:'+str(zoomed_image.shape))
axs[2].imshow(scaled_image)
axs[2].set_title('Scaled Image Shape:'+str(scaled_image.shape))
```

Image Rotation

Rotates image around center point by specified angle (30°)

Process: Define rotation matrix → Apply warp affine transformation → Display result

Formula: Rotation Matrix = [[cos(θ), -sin(θ)], [sin(θ), cos(θ)]]

```python
img = cv2.imread('Images/Task 5.jpg')
image_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
center = (image_rgb.shape[1] // 2, image_rgb.shape[0] // 2)
angle = 30
scale = 1
rotation_matrix = cv2.getRotationMatrix2D(center, angle, scale)
rotated_image = cv2.warpAffine(image_rgb, rotation_matrix, (img.shape[1], img.shape[0]))

fig, axs = plt.subplots(1, 2, figsize=(7, 4))
axs[0].imshow(image_rgb)
axs[0].set_title('Original Image')
axs[1].imshow(rotated_image)
axs[1].set_title('Image Rotation')
for ax in axs:
    ax.set_xticks([])
    ax.set_yticks([])

plt.tight_layout()
plt.show()
```

✓ 0.0s

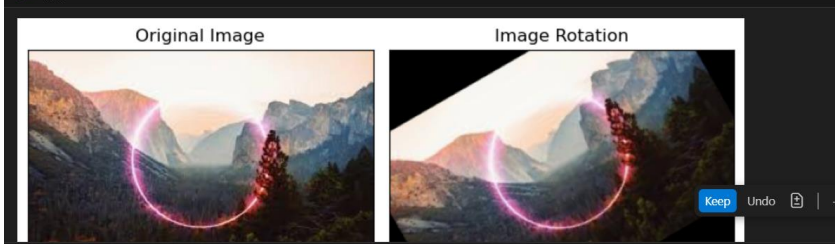


Image Translation

Shifts image by specified pixels (100px horizontal, 70px vertical)

Translation matrix: [[1, 0, tx], [0, 1, ty]]Four directions demonstrated: Left (-50), Right (+50), Top, Bottom

```python
img = cv2.imread('Images/Task 5.jpg')
image_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
width, height = image_rgb.shape[1], image_rgb.shape[0]

tx, ty = 100, 70
translation_matrix = np.array([[1, 0, tx], [0, 1, ty]], dtype=np.float32)
translated_image = cv2.warpAffine(image_rgb, translation_matrix, (width, height))

fig, axs = plt.subplots(1, 2, figsize=(7, 4))
axs[0].imshow(image_rgb), axs[0].set_title('Original Image')
axs[1].imshow(translated_image), axs[1].set_title('Image Translation')

for ax in axs:
    ax.set_xticks([]), ax.set_yticks([])

plt.tight_layout()
plt.show()
```

✓ 0.0s

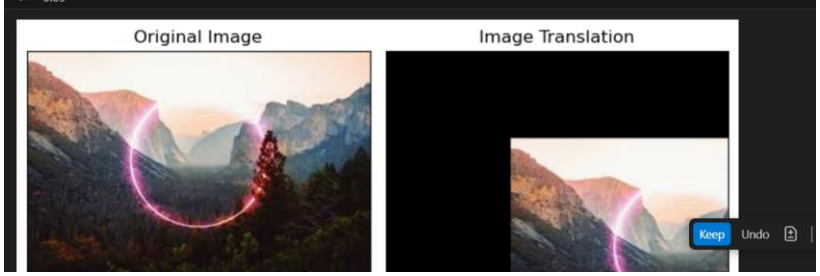


Image Shearing

Applies shear transformation to distort image (shearX = -0.15)

Shear matrix: [[1, shearX, 0], [0, 1, shearY]]

```python
image = cv2.imread('Images/Task 5.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
width, height = image_rgb.shape[1], image_rgb.shape[0]

shearX, shearY = -0.15, 0
transformation_matrix = np.array([[1, shearX, 0], [0, 1, shearY]], dtype=np.float32)
sheared_image = cv2.warpAffine(image_rgb, transformation_matrix, (width, height))

fig, axs = plt.subplots(1, 2, figsize=(7, 4))
axs[0].imshow(image_rgb), axs[0].set_title('Original Image')
axs[1].imshow(sheared_image), axs[1].set_title('Sheared Image')

for ax in axs:
    ax.set_xticks([]), ax.set_yticks([])

plt.tight_layout()
plt.show()
```

✓ 0.1s

1.4 Edge Detection

Edge detection identifies boundaries between regions with different intensities. Essential for object detection and image segmentation.

Canny Edge Detection

Multi-stage algorithm for robust edge detection:

1. Gaussian blur to reduce noise

2. Gradient calculation using Sobel operator

3. Non-maximum suppression to thin edges

4. Double threshold hysteresis to classify edges

Parameters: Lower threshold=100, Upper threshold=700, Aperture size=3

Output: Binary image with white edges on black background

1.5 Image Normalization

Normalization scales pixel values to a standard range [0, 1] or [0, 255]. Important for consistent processing and machine learning models.

Formula: Normalized = (value - min) / (max - min)

Benefits: Improves algorithm convergence, standardizes input ranges, enhances contrast

Method: Split RGB channels → Normalize each channel → Merge channels

```
image = cv2.imread('Images/Task 5.jpg')
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
kernel = np.ones((3, 3), np.uint8)

dilated = cv2.dilate(image_gray, kernel, iterations=2)
eroded = cv2.erode(image_gray, kernel, iterations=2)
opening = cv2.morphologyEx(image_gray, cv2.MORPH_OPEN, kernel)
closing = cv2.morphologyEx(image_gray, cv2.MORPH_CLOSE, kernel)

fig, axs = plt.subplots(2, 2, figsize=(7, 7))
axs[0, 0].imshow(dilated, cmap='Greys'), axs[0, 0].set_title('Dilated Image')
axs[0, 1].imshow(eroded, cmap='Greys'), axs[0, 1].set_title('Eroded Image')
axs[1, 0].imshow(opening, cmap='Greys'), axs[1, 0].set_title('Opening')
axs[1, 1].imshow(closing, cmap='Greys'), axs[1, 1].set_title('Closing')

for ax in axs.flatten():
    ax.set_xticks([]), ax.set_yticks([])

plt.tight_layout()
plt.show()
```

2. IMAGE SEGMENTATION AND THRESHOLDING

Image segmentation partitions an image into meaningful regions. Thresholding is a fundamental segmentation technique that converts grayscale images to binary images based on intensity values. This section covers simple, adaptive, and automatic thresholding methods.

```
image = cv2.imread('Images/Task 5.jpg')
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
✓ 0.0s

def show_image(img, title):
    plt.imshow(img, cmap='gray')
    plt.title(title)
    plt.axis('off')
    plt.show()
✓ 0.0s

show_image(gray_image, 'Original Grayscale Image')
✓ 0.0s
```

2.1 Simple Thresholding Methods

Simple thresholding applies a single threshold value: pixels above threshold become white (255), pixels below become black (0). Threshold value = 120.

Binary

Pixels > T → 255, Pixels ≤ T → 0

Binary Inverted

Pixels > T → 0, Pixels ≤ T → 255 (opposite of binary)

Truncated

Pixels > T → T, Pixels ≤ T → 0 (cap at threshold)

To Zero

Pixels > T → Pixel value, Pixels ≤ T → 0

To Zero Inverted

Pixels > T → 0, Pixels ≤ T → Pixel value

Figure 2.1: Contrast Stretched Image (Piecewise-linear transformation)

2.2 Adaptive Thresholding

Adaptive thresholding calculates different threshold values for different regions of the image. Useful when illumination varies across the image.

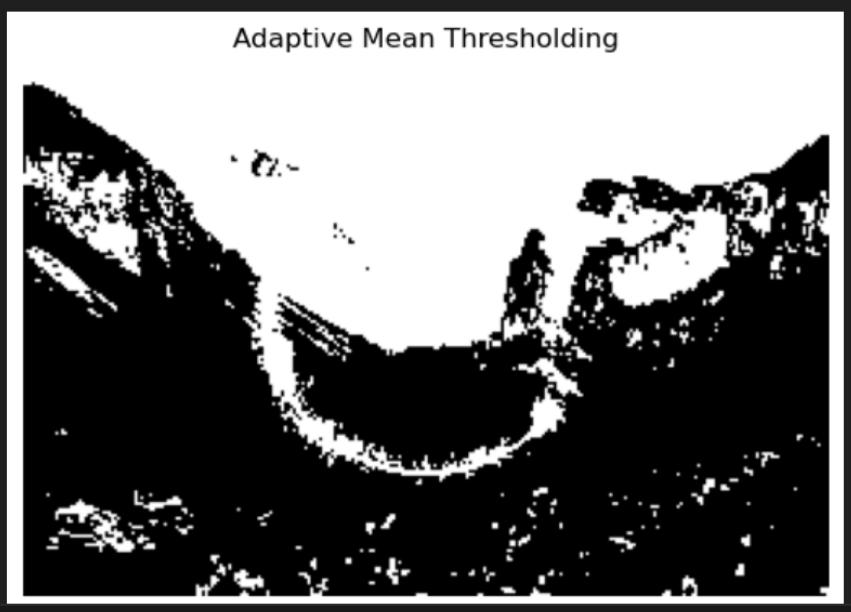
Adaptive Mean Thresholding

Threshold for each pixel = mean of 199×199 neighborhood - constant (5)

Benefits: Handles varying lighting conditions, preserves local details

```
thresh_mean = cv2.adaptiveThreshold(
    gray_image, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    199, 5
)
show_image(thresh_mean, "Adaptive Mean Thresholding")
```

✓ 0.0s



Spaces: 4

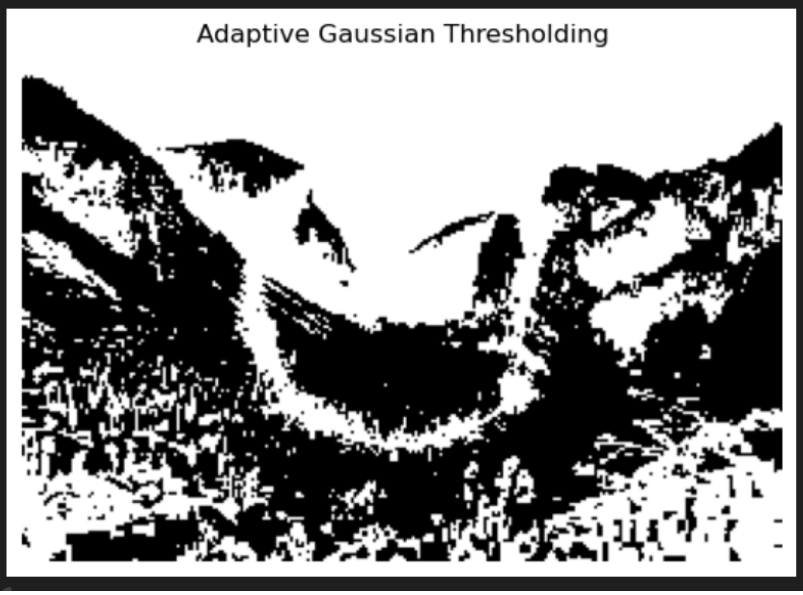
Adaptive Gaussian Thresholding

Threshold for each pixel = weighted sum (Gaussian window) of 199×199 neighborhood - constant (5)

Benefits: More sophisticated weighting, better for complex lighting variations

```python
thresh_gauss = cv2.adaptiveThreshold(
    gray_image, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    199, 5
)
show_image(thresh_gauss, "Adaptive Gaussian Thresholding")
```

✓ 0.0s



2.3 Otsu Thresholding

Otsu thresholding automatically calculates optimal threshold value by minimizing intra-class variance. No manual threshold value needed.

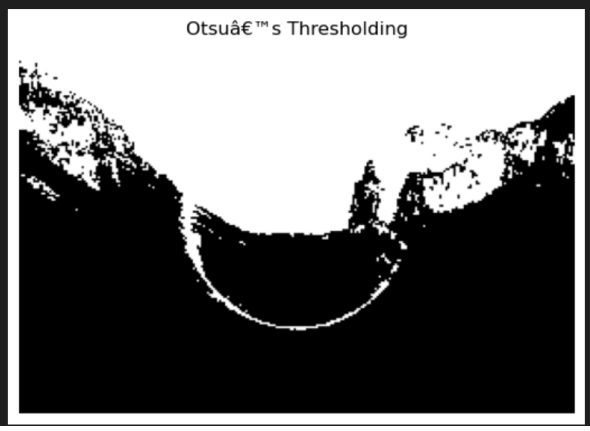
Automatic calculation of optimal threshold from image histogram

Advantages: No parameter tuning required, works well for bimodal distributions, optimal for foreground-background separation

```python
ret, otsu_thresh = cv2.threshold(
    gray_image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Calculated Otsu threshold value:", ret)
show_image(otsu_thresh, "Otsuâ€™s Thresholding")
```

✓ 0.0s

```
Calculated Otsu threshold value: 129.0
```



3. MORPHOLOGICAL OPERATIONS & FILTERING

Morphological operations process images based on their shapes. They use a structuring element (kernel) to probe and reshape the image. Combined with filtering techniques, these operations enhance image quality and extract useful features.

3.1 Erosion and Dilation

Erosion

Erosion shrinks white regions by moving the kernel over the image. A pixel becomes white only if all pixels under the kernel are white.

Kernel: 5×5 matrix of ones

Iterations: 1

Effects: Removes small white objects, separates connected objects

Dilation

Dilation expands white regions. A pixel becomes white if any pixel under the kernel is white.

Kernel: 5×5 matrix of ones

Iterations: 1

Effects: Fills small holes, connects nearby objects

```
img_erosion = cv2.erode(img, kernel, iterations=1)

plt.imshow(img_erosion, cmap='gray')
plt.title("After Erosion")
plt.axis('off')
plt.show()
```

✓ 0.0s

3.2 Opening and Closing

Opening

Opening = Erosion followed by Dilation

Effects: Removes small objects and noise while preserving larger structures

Use case: Background noise removal

Closing

Closing = Dilation followed by Erosion

Effects: Fills holes in objects, connects nearby objects

Use case: Object completion, filling gaps

3.3 Bilateral Filtering

Bilateral filter applies edge-preserving smoothing using spatial and intensity differences.
Parameters: d=15, sigmaColor=75, sigmaSpace=75

Output saved as: taj_bilateral.jpg

3.4 Image Denoising

Non-local means denoising uses similarity between image patches for noise reduction.
cv2.fastNlMeansDenoisingColored() applies this to color images.

Parameters: h=10 (filter strength), templateWindowSize=10, searchWindowSize=15

Advantages: Preserves textures, maintains fine details while removing noise

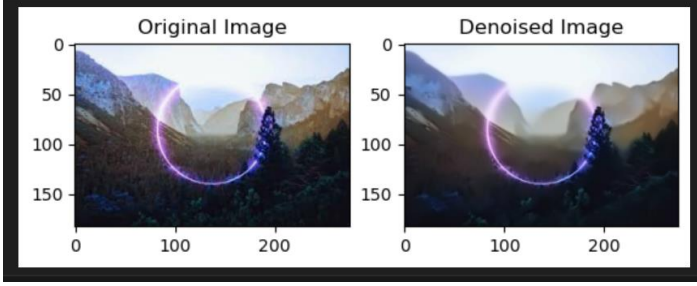
```python
# Importing required libraries
import numpy as np
import cv2
from matplotlib import pyplot as plt

# Reading the noisy image from file
img = cv2.imread('Images/Task 5.jpg')

# Applying denoising filter
dst = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 15)

# Displaying original and denoised images
plt.subplot(121), plt.imshow(img), plt.title('Original Image')
plt.subplot(122), plt.imshow(dst), plt.title('Denoised Image')
plt.show()
```

✓ 0.2s



4. ADVANCED IMAGE MANIPULATION & BACKGROUND SUBTRACTION

Advanced techniques extract foreground objects, remove backgrounds, and handle complex image analysis. These methods are crucial for video analysis, object tracking, and image segmentation.

4.1 Image Inpainting

Image inpainting reconstructs missing or damaged image regions using surrounding information. First, a mask is created marking regions to inpaint.

Process: Create binary mask → Apply inpainting algorithm → Fill damaged regions

Mask saved as: mask.jpg

Figure 4.1: Inpainting Mask (white=regions to inpaint)

Algorithm: Reconstructs pixels based on texture and structure from boundary pixels

4.2 Background Subtraction

Background subtraction identifies moving foreground objects in video sequences. MOG2 (Mixture of Gaussians) models each pixel as mixture of Gaussian distributions.

Advantages: Handles illumination changes, shadows, moving backgrounds

Process: Load video → Initialize MOG2 → Apply for each frame → Extract foreground mask

Output: Binary mask showing foreground objects (white) on black background

```python
import numpy as np
import cv2

# Load video file
cap = cv2.VideoCapture('Videos/Task 5.mp4')

# Create background subtractor (MOG2 handles shadows well)
fgbg = cv2.createBackgroundSubtractorMOG2()

while True:
    ret, frame = cap.read()
    if not ret:
        break  # Stop if video ends

    # Apply background subtraction
    fgmask = fgbg.apply(frame)

    # Show original and foreground mask side by side
    cv2.imshow('Original Frame', frame)
    cv2.imshow('Foreground Mask', fgmask)

    # Press 'Esc' to exit
    if cv2.waitKey(30) & 0xFF == 27:
        break

# Release resources
cap.release()
cv2.destroyAllWindows()
```

4.3 Foreground Extraction - GrabCut Algorithm

GrabCut performs interactive image segmentation using graph cuts. Requires approximate bounding rectangle around foreground object.

Algorithm: Graph-based optimization for optimal foreground/background labeling

Parameters: Rectangle=(20, 100, 150, 150), Iterations=3

Advantages: Interactive refinement possible, very accurate segmentation

5. FEATURE DETECTION AND DESCRIPTION

Feature detection identifies distinctive keypoints in images. These features are used for image matching, object recognition, tracking, and 3D reconstruction. This section covers lines and corner detection techniques.

5.1 Line Detection Hough Lines

Hough Line Transform detects straight lines in binary edge images. Robust to noise and partial occlusion.

Algorithm Steps

1. Read original image and convert to grayscale

2. Apply Canny edge detection (50-150 thresholds)

3. Apply Hough Lines transform ($\rho=1$, $\theta=\pi/180$, threshold=100)

4. Draw detected lines on original image (red lines)

5. Save and display result with line count

Parameters

ρ (rho): Distance resolution in pixels = 1

θ (theta): Angle resolution in radians = $\pi/180 \approx 1$ degree

threshold: Minimum votes to detect a line = 100

Output: linesDetected.jpg - Original image with detected lines drawn in red

[Image not found: linesDetected.jpg]

5.2 Corner Detection

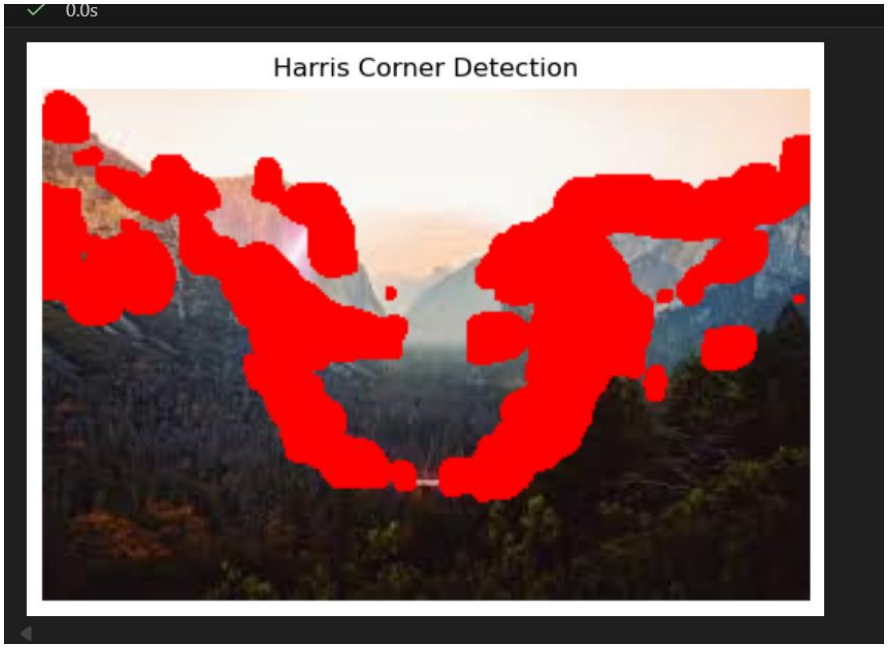
Corner detection identifies sharp intensity changes (corners) in images. Used for feature matching, image alignment, and object tracking.

Harris Corner Detection

Harris detector responds to corner features where gradient changes significantly in multiple directions.

Formula: Response = $\det(M) - k \times \mathrm{trace}(M)^2$, where M is Harris matrix

Advantages: Accurate, computationally efficient



Shi-Thomasi Corner Detection

Improved Harris method using min eignevalue instead of Harris response.

Formula: Response = min($eigenvalue_1$, $eigenvalue_2$)

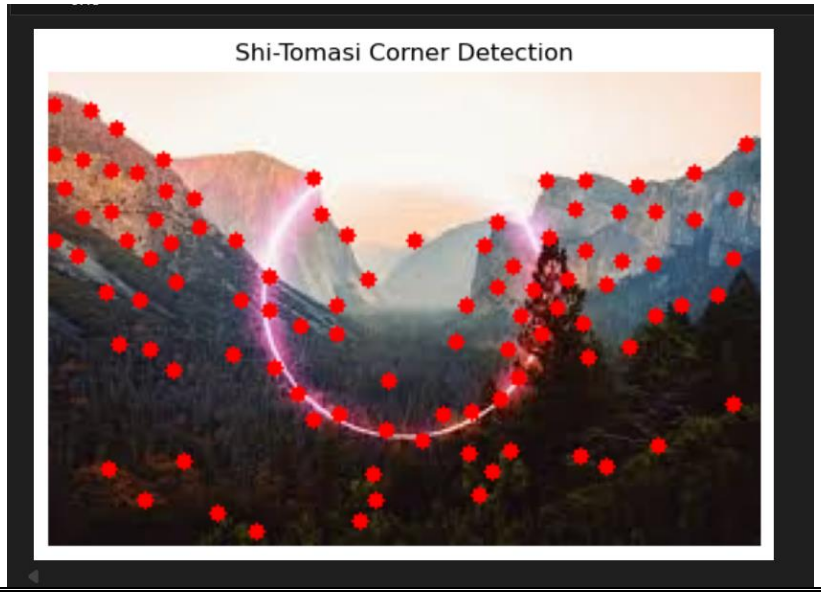
Advantages: Better tracking performance, more stable for feature matching

Good Features to Track

Combines corner detection with tracking capability. Automatically selects strongest corners.

Parameters: maxCorners=27, qualityLevel=0.01, minDistance=10

Output: Sub-pixel corner locations for precise tracking



6. SUMMARY AND APPLICATIONS

This comprehensive task demonstrates the full pipeline of image processing with OpenCV, covering fundamental techniques to advanced algorithms.

Key Techniques Covered

Image Enhancement: Blur, denoise, normalize for improved quality

Image Segmentation: Separate foreground/background using thresholding

Morphological Operations: Shape-based image processing and analysis

Object Detection: Line and corner detection for feature extraction

Video Analysis: Background subtraction and foreground extraction

Practical Applications

Medical Imaging: Tumor detection and organ segmentation

Autonomous Vehicles: Lane detection and obstacle identification

Object Recognition: Feature extraction for ML/DL models

Video Surveillance: Motion detection and tracking

Document Processing: Text extraction and analysis

Industrial Inspection: Quality control and defect detection